The Nelson–WC Boundary: A Quantitative Analysis of Internal Soundness in Recursive Arithmetic

Abstract

We formalize, in Agda with `--safe`, a quantitative boundary between Nelson-style proof-by-proof internal soundness and Gödel-style obstruction in a system of recursive arithmetic over binary trees, following Rose [1]. The system includes a 26-rule equational derivation calculus, a tree-recursive proof checker (thFunTFor), and a ground evaluator (evalCode). We define a well-chained certificate type WC with 16 constructors covering all 26 rules, each carrying explicit local side conditions, and prove semantic soundness: WC $p \to$ SemTrue p. Every genuine derivation yields a WC certificate once the relevant EvalOK obligations are supplied. We show that EvalOK cannot hold uniformly: an EvalOK instance for the false equation $O = \mathsf{Pair}(O, O)$ implies inconsistency. We identify a zero-backtracking fragment WC_0 where all side conditions discharge automatically, and show this fragment includes the identity axiom, reflexivity, symmetry, transitivity, unary congruence, and Schema F. The obstruction to universalization is localized to a specific computational parameter—the backtracking level of semantic verification—connecting Nelson's program in *Elements* [3] to the game-semantic complexity analysis of Aschieri [4].

1 The System

We work in a purely equational theory over binary trees (*lf* and *nd a b*), with terms O, *var n*, *ap1 f t*, *ap2 g* t_1 t_2, 1-ary functors I, *Fst*, *Snd*, *Comp*, *Comp2*, *Rec*, KT, and 2-ary functors Pair, *Const*, *Lift*, *Post*, *Fan*, *IfLf*, TreeEq.

The derivation system Deriv *hyp eq* is an inductive type with 26 rules: 18 computation axioms (including *axI*, *axFst*, *axRecLf*, *axRecNd*, *axTreeEqLL/LN/NL/NN*, *axIfLfL/N*, etc.), structural rules (*axRefl*, *ruleSym*, *ruleTrans*, *cong1*, *congL*, *congR*), variable instantiation (*ruleInst*), hypothesis introduction (*ruleHyp*), and Schema F (uniqueness of tree recursion).

Schema F says: if f and g both satisfy the same recursion equation (*Rec z s*), then $f(x) = g(x)$ for all x. This replaces induction and is what makes the system strong enough for Gödel II.

The system includes:

- A coding function code : *Term* $\to$ *Tree* with reify : *Tree* $\to$ *Term*.
- A proof checker thFunTFor : *Fun1*, defined as *Rec O thFunStep*, where *thFunStep* is a 26-branch dispatch.
- A ground evaluator evalCode : *Fun1*, defined as *Rec O evalStep*, dispatching on tag-O (returns O), tag-*Ap1* (strips the functor), and default (passthrough).

- A coded substitution $\mathsf{substTFor} : \mathit{Fun1}$, implementing variable replacement on coded terms.

Consistency is: $\mathsf{Consistent}\ \mathit{hyp} = \mathsf{Deriv}\ \mathit{hyp}\ (O = \mathsf{Pair}(O, O)) \to \bot$.

2 Gödel's Second Incompleteness Theorem

Following Rose [1], we construct a Gödel sentence via diagonal substitution. The template equation is

$$\mathsf{TreeEq}(\mathsf{thFunTFor}(x),\ \mathsf{substTFor}(y)) = \mathsf{Pair}(O, O)$$

with free variables x (proof encoding) and y (coded term). Since $\mathsf{TreeEq}(a, b) = O$ iff $a = b$ and $\mathsf{TreeEq}(a, b) = \mathsf{Pair}(O, O)$ iff $a \neq b$, this says: "the equation proved by encoding x does not match the coded substitution result."

The Gödel sentence G is obtained by substituting the code of the template itself for y, yielding (after closing) a fixed-point: G says "no proof encoding proves G."

Theorem 1 (Gödel II, `GodelII.agda`)**.**

$$\mathsf{ProofE}\ \mathit{hyp}\ \to\ \mathsf{Consistent}\ \mathit{hyp}\ \to\ \mathsf{Deriv}\ \mathit{hyp}\ G\ \to\ \bot$$

The proof uses *thm14E* (every derivation has a correct proof encoding), the diagonal construction (coded substitution closes to a fixed point), and $\mathsf{TreeEq}(t, t) = O$ (proved via Schema F, not metalevel computation). From $\mathsf{Deriv}\ \mathit{hyp}\ G$ one derives both $\mathsf{TreeEq}(X, X) = \mathsf{Pair}(O, O)$ and $\mathsf{TreeEq}(X, X) = O$ for the same X, giving $O = \mathsf{Pair}(O, O)$.

Remark 2. *Schema F was corrected during this development. The original formulation required proving $f(\mathsf{Pair}(a, b)) = g(\mathsf{Pair}(a, b))$ without inductive hypothesis. The corrected version requires f and g to satisfy the* same *recursion equation, matching Rose's original Schema F (uniqueness of the fixed point of tree recursion).*

3 The Well-Chained Fragment

Definition 3. *The* semantic truth predicate *is:*

$$\mathsf{SemTrue}(p, \mathit{hyp})\ =\ \mathsf{Deriv}\ \mathit{hyp}\ (\mathsf{evalCode}(\mathsf{Fst}(\mathsf{thFunTFor}(p)))\ =\ \mathsf{evalCode}(\mathsf{Snd}(\mathsf{thFunTFor}(p))))$$

The evaluation side condition *for $t = u$ is:*

$$\mathsf{EvalOK}(t, u, \mathit{hyp})\ =\ \mathsf{Deriv}\ \mathit{hyp}\ (\mathsf{evalCode}(\mathsf{reify}(\mathsf{code}(t)))\ =\ \mathsf{evalCode}(\mathsf{reify}(\mathsf{code}(u))))$$

Definition 4. $\mathsf{WC}\ p\ \mathit{hyp}$ *is an inductive type with 16 constructors covering all 26 derivation rules. Each constructor carries a* Nelson proof *(the* $\mathsf{thFunTFor}$ *reduction for that rule) and* local side conditions.

The side conditions fall into three classes:

1. ***None*** *(axRefl, ruleSym): semantic truth follows from the induction hypothesis or is trivial.*

2. ***Matching*** *(ruleTrans): the middle term of the two sub-proofs must agree.*

3. ***Evaluation*** *(base cases, ruleInst, congL/R): an* EvalOK*-type condition that depends on the specific proof and cannot be derived from the IH.*

A generic constructor wcBase *handles any rule given its Nelson proof and evaluation condition.*

4 The Nelson–WC Boundary

Theorem 5 (Local Soundness, `WellChained.agda`)**.** $\mathsf{WC}\ p\ hyp \Rightarrow \mathsf{SemTrue}(p, hyp)$.

Theorem 6 (Completeness with Side Conditions, `NelsonWCBoundary.agda`)**.** *For every* $d : \mathsf{Deriv}\ hyp\ (t = u)$ *and hypothesis certificate* $ph : \mathsf{ProofE}\ hyp$*, if* $\mathsf{EvalOK}(t, u, hyp)$ *is supplied, then there exists* $\mathsf{enc}(d)$ *with* $\mathsf{WC}\ \mathsf{enc}(d)\ hyp$.

Theorem 7 (Obstruction, `NelsonWCBoundary.agda`)**.** $\mathsf{EvalOK}(O,\ \mathsf{Pair}(O, O),\ hyp) \Rightarrow \mathsf{Consistent}\ hyp \Rightarrow \bot$.

Proof. $\mathsf{evalCode}(\mathsf{reify}(\mathsf{code}(O))) = O$ by the tag-O case. $\mathsf{evalCode}(\mathsf{reify}(\mathsf{code}(\mathsf{Pair}(O, O))))$ is a Pair: the code of $\mathsf{Pair}(O, O)$ has tag tagAp2, which misses both tag-O and tag-Ap1 checks, triggering passthrough. So EvalOK gives $O = \mathsf{Pair}(A, B)$ for specific A, B. Then $\mathsf{TreeEq}(O, O) = O$ and $\mathsf{TreeEq}(O, \mathsf{Pair}(A, B)) = \mathsf{Pair}(O, O)$ by the TreeEq axioms, giving $O = \mathsf{Pair}(O, O)$ by transitivity. □

Corollary 8 (The Nelson–WC Boundary)**.**

1. *For each genuine derivation* d*, the* EvalOK *obligation can be discharged (it reduces to a concrete computation on coded terms).*

2. *But* EvalOK *cannot hold uniformly, since it would then hold for false equations, contradicting consistency.*

3. *Therefore* WC *certificates and* SemTrue *cannot be internalized as provable universal statements.*

5 The Zero-Backtracking Fragment

Definition 9. $\mathsf{WC}_0\ p\ hyp$ *is a restricted inductive type with 7 constructors:* wc0Refl*,* wc0AxI*,* wc0Sym*,* wc0Trans*,* wc0TransO*,* wc0Cong1*,* wc0RuleF*. No constructor carries external* EvalOK *obligations.*

The key lemma enabling this:

Theorem 10 (Universal Functor Miss, `BoundaryQuantitative.agda`)**.** *For every Fun1* f*:*

$$\mathsf{TreeEq}(\mathsf{reify}(\mathsf{codeF1}\ f),\ O) = \mathsf{Pair}(O, O)$$
$$\mathsf{TreeEq}(\mathsf{reify}(\mathsf{codeF1}\ f),\ \mathsf{tagAp1T}) = \mathsf{Pair}(O, O)$$

Consequently, $\mathsf{evalCode}(\mathsf{mkAp1}(\mathsf{codeF1}\ f,\ t)) = \mathsf{evalCode}(t)$ *for all* f *and all* t.

Proof. By case analysis on f (7 cases: I, *Fst*, *Snd*, *Comp*, *Comp2*, *Rec*, KT). Each $\mathsf{codeF1}\ f$ has the form $nd(\mathsf{natCode}(26{+}k), payload)$, so reify gives $\mathsf{Pair}(\mathsf{Pair}(O, \ldots), \ldots)$, which misses both tag checks. □

Theorem 11 (Zero-Backtracking Soundness, `BoundaryQuantitative.agda`)**.**

$$\mathsf{WC}_0\ p\ hyp\ \Rightarrow\ \mathsf{SemTrue}(p, hyp)$$

with ***no external side conditions****.*

Proof. By induction on WC_0. The axI and axKT cases use the universal functor miss lemma. The $\mathsf{cong1}$ case uses the fact that any $\mathsf{Pair}(\mathsf{Pair}(f_1, f_2), f_3)$ automatically passes both miss checks (by $\mathsf{axTreeEqNL}$ and the IfLf argument). The structural cases (Sym, Trans) compose from the induction hypothesis. □

6 The Quantitative Boundary

The WC_0 fragment is the *zero-backtracking* layer of a natural stratification:

Definition 12. *The* backtracking level BLevel *of a* WC *certificate is:*

- $\mathsf{BLevel} = 0$ *for base cases and structural rules without ruleInst.*
- $\mathsf{BLevel} = k{+}1$ *for* $\mathsf{wcRuleInst}$ *with a sub-proof of level* k*.*
- $\mathsf{BLevel} = \max(\mathsf{BLevel}_1, \mathsf{BLevel}_2)$ *for binary rules.*

The side condition for *ruleInst* has the form:

$$\mathsf{evalCode}(\mathsf{substTFor}(L)) = \mathsf{evalCode}(\mathsf{substTFor}(R))$$

where $\mathsf{substTFor}$ does not commute with $\mathsf{evalCode}$. Each nested *ruleInst* adds a layer of $\mathsf{substTFor}$ that must be verified independently.

The quantitative boundary:

$\mathsf{BLevel} = 0$	free	automatic (WC_0, this paper)
$\mathsf{BLevel} = k$	k-fold $\mathsf{substTFor}$ verification	per-instance
$\mathsf{BLevel} = \infty$	impossible	Theorem 7

Every concrete derivation has some finite BLevel, but no single internal theorem proves $\mathsf{SemTrue}$ for all levels simultaneously.

7 Connection to Nelson's Program

In *Elements* [3], Nelson proposes verifying mathematical proofs "from below": for each specific proof, checking that the needed bounds hold, rather than establishing them universally. He emphasizes that the bounds depend on structural parameters (rank and level) rather than on proof length.

Our formalization makes this precise:

- The WC type is the certificate format.
- The explicit side conditions are Nelson's "needed bounds."
- semSound is the local verification.
- The obstruction theorem says exactly why the local strategy cannot be globalized.

Nelson's key insight in *Elements* §14 concerns the predicate $\varepsilon(y) \leftrightarrow \forall x[x \uparrow y \text{ terminates}]$. For each specific string X, the system S^* proves $\varepsilon(*X)$. But S^* cannot prove $\forall x\, \varepsilon(x)$. The gap between "for each specific X" and "for all X" is exactly our gap between "for each derivation, EvalOK is satisfiable" and "EvalOK cannot hold uniformly." Nelson's ε is the exponentiation-totality predicate; our EvalOK is the evaluation-agreement predicate. Both are local conditions that work proof-by-proof but resist universalization.

8 Connection to Aschieri's Backtracking Analysis

Aschieri [4] shows that the complexity of cut-elimination and strategy interaction is governed not by crude proof depth but by the *backtracking level*—the depth of mind-change chains in the associated game-semantic strategy. His Max-Min Theorem says: the real interaction complexity between an n-backtracking and an m-backtracking strategy is controlled by $\min(n, m)$, not $\max(n, m)$.

Our BLevel stratification has a strikingly parallel form:

- BLevel $= 0$: internally manageable, no external obligations (like 1-backtracking strategies: safe in any context).
- Complexity grows by controlled layers: each increment of BLevel corresponds to one extra substTFor verification.
- The uniform "all levels at once" claim breaks (like trying to bound all backtracking levels simultaneously).

Conjecture 13 (Nelson–Aschieri Connection)**.** *There exists a finite-level hierarchy* BLevel_k *of semantic side conditions such that:*

1. BLevel_0 *corresponds to a zero-backtracking, obligation-free fragment (*WC_0*);*
2. *each increment of k corresponds to one extra layer of semantic backtracking through substitution;*
3. *every concrete derivation has some finite k;*
4. *no single internal theorem proves* BLevel_k *for all k.*

Moreover, BLevel_k *is a Nelson-side semantic analogue of Aschieri's game-semantic backtracking level.*

The salvaged version of Nelson's program is then:

> Internal soundness certification is a stratified capability. The system can verify itself at every finite backtracking level, with costs controlled by proof structure, not proof length. The inability to do it uniformly is the incompleteness theorem in computational form.

9 What Is New

The result is not a restatement of Gödel II. Gödel II says: a consistent system cannot prove its own consistency. We say something more specific:

> The exact computational locus of incompleteness is the side condition EvalOK needed to turn decoded proof outputs into semantic truth.

The obstruction is not:

- diagonalization (the diagonal is constructed successfully);
- self-reference (thFunTFor works correctly on all inputs);
- recursion or complexity (all functions terminate; all proofs check in under 1 second);
- proof encoding (ProofE exists for every derivation).

The obstruction *is*: evalCode, which evaluates coded terms by tree-structural tag dispatch, cannot agree on the two sides of an arbitrary coded equation. Concretely:

- evalCode dispatches on Fst of its input, conflating code structure with data structure (Pair(O, b) triggers tag-O).
- substTFor does not commute with evalCode: the *ruleInst* side condition cannot be derived from the IH.
- Any ground evaluator faces the same tension: to validate base-case axioms it must evaluate functors; but evaluating functors on coded terms requires distinguishing code-level tags from data-level structure, which cannot be done uniformly within the system's own equational theory.

10 Formalization

All results are in the `Guard/` directory, verified with Agda 2.9.0 and `--safe` (no postulates, no `--type-in-type`, no `--with-K`).

`WellChained.agda` (475 lines)	WC type + semSound
`BoundaryQuantitative.agda` (293 lines)	WC_0 + semSound0
`NelsonWCBoundary.agda` (233 lines)	Bridge + obstruction
`Thm14E.agda` (1402 lines)	Deriv $\to$ ProofE
`GodelII.agda`	Gödel II
`NelsonDispatch.agda` (595 lines)	All 26 tag dispatches
`EvalCode*.agda`	Ground evaluator + lemmas
`Nelson*.agda`	Per-rule Nelson proofs (13+ tags)
`NelsonPipeline.agda` (277 lines)	Pipeline demonstrations

References

[1] H. E. Rose, *On the Consistency and Undecidability of Recursive Arithmetic*, Zeitschr. f. math. Logik und Grundlagen d. Math., Bd. 7, S. 124–135 (1961).

[2] E. Nelson, *Predicative Arithmetic*, Mathematical Notes 32, Princeton University Press, 1986.

[3] E. Nelson, *Elements*, arXiv:1510.00369 [math.LO], 2015 (unfinished, with afterword by S. Buss and T. Tao).

[4] F. Aschieri, *Game Semantics and the Geometry of Backtracking: a New Complexity Analysis of Interaction*, arXiv:1511.06260v3 [math.LO], 2016.

[5] S. Kritchman and R. Raz, *The Surprise Examination and the Second Incompleteness Theorem*, Notices of the AMS, **57**, 1454–1458, 2010.

[6] J. Guard, *Lecture Notes on Recursive Arithmetic*, 1963.